

OpenDeepBench and OpenDeepAPM: Deep Infrastructure Benchmarking and Trace-Correlated Observability for Multi-Cloud LLM Inference

Srikanta Datta Tumkur*, Anshu Bansal*, Mehar Simhadri[†], Jay Iyer[†]

*MIT Sloan School of Management, Cambridge, MA – {srikanta, anshu.bansal}@sloan.mit.edu

[†]IEEE Senior Members – {meher.simhadri, jay.iyer}@ieee.org

Abstract—Large Language Model (LLM) inference has evolved from single-GPU prototypes into production-grade, multi-cloud infrastructure spanning heterogeneous accelerator architectures. Yet the benchmarking and observability ecosystems remain critically fragmented: MLPerf Inference measures raw throughput on fixed models in single-cloud settings; traditional APM tools designed for deterministic microservices fail catastrophically for non-deterministic LLM inference; and no platform today provides end-to-end trace correlation from user traffic through GPU silicon to root cause. We present two complementary systems. OpenDeepAPM is a deep LLM Application Performance Monitoring architecture that constructs unified traces spanning 27 linked spans across seven layers—from client traffic and CPU preprocessing through GPU pre-fill/decode phases, per-rank tensor-parallel execution, per-layer attention and MLP operations, NCCL/NVLink/InfiniBand collectives, KV-cache lifecycle, and storage—with silicon metrics (SM utilization, tensor core activity, HBM bandwidth, L2 hit rates) attached as evidence to each span via NVTX-linked CUDA timelines and DCGM counters. The correlation engine joins traces using composite keys (trace_id + time_window + node_id + gpu_uuid + rank_id + nccl_comm_id + nic_id) across a time-series graph database to produce automated root-cause attribution. OpenDeepBench is a multi-cloud, multi-silicon, multi-model LLM infrastructure benchmark that evaluates 16 dimensions—cloud provider, region, silicon vendor, accelerator model, serving engine, runtime, collective backend, model size, context size (1K–128K), output length, concurrency, batch policy, KV-cache mode, attention type, network topology, and storage—producing a normalized scorecard of TTFT, TPOT, throughput, cost/token, energy/token, and reliability with automated root-cause diagnosis. Across 11 cloud providers, 6 accelerator families, and 12 benchmark suites (ODB-Chat through ODB-Reliability), OpenDeepBench reveals: (1) 3.2× throughput variance for identical models across clouds; (2) KV-cache memory bandwidth as the dominant decode bottleneck with TPOT degrading sharply beyond 32K context; (3) AMD MI355X achieving 97% of NVIDIA B200 decode performance at 35% lower cost; (4) inter-node fabric congestion as the primary source of p99 latency spikes in multi-node tensor-parallel deployments; and (5) cross-cloud disaggregated inference reducing cost 15–29% at iso-latency. We detail the OpenTelemetry integration architecture, time-series graph correlation engine, compliance framework, and propose the OpenDeepBench Composite Score as a new industry standard.

Index Terms—LLM inference, deep APM, distributed trac-

ing, OpenTelemetry, multi-cloud benchmarking, GPU observability, KV-cache, NCCL, NVLink, InfiniBand, root-cause analysis, time-series graph database

I. Introduction

The AI inference landscape has entered a new era. Global spending on AI infrastructure exceeded \$200B in 2025, yet organizations deploying LLMs in production face a dual paradox: the benchmarks used to select hardware measure almost none of the metrics that determine real-world performance, and the monitoring tools used to operate inference infrastructure were designed for a fundamentally different class of workloads.

Gap 1: The Benchmark-Production Disconnect. MLPerf Inference v6.0 [1] evaluates raw throughput on fixed models in controlled, single-cloud environments. Production inference deployments include guardrails [2], PII masking, watermarking [3], and compliance logging that add 18–61% latency. No existing benchmark answers: “Which cloud + GPU + serving engine + context size gives the best TTFT, TPOT, throughput, cost/token, and reliability under realistic multi-tenant traffic?”

Gap 2: The Observability Void. Traditional APM tools (Datadog, New Relic, Dynatrace) instrument deterministic microservices with request-response spans. LLM inference is fundamentally different: a single request generates a trace containing dozens of linked spans across traffic ingress, CPU tokenization, GPU scheduling, tensor-parallel prefill across 16 ranks on 2 nodes, 80 transformer layers per phase, NCCL collectives over NVLink and InfiniBand, paged KV-cache allocation and lifecycle, autoregressive decode with growing context, and response streaming. No existing platform correlates GPU thermal throttling with output quality degradation, or KV-cache eviction with downstream accuracy loss, or NCCL stalls with per-token latency spikes.

Gap 3: Architecture Blind Spots. NVIDIA-centric benchmarks systematically favor compute-bound workloads. AMD Instinct MI355X [4] with 288 GB HBM3E and Intel Gaudi 3 [5] with cost-optimized matrix engines

excel in memory-bound and energy-constrained regimes that current benchmarks underrepresent.

This paper addresses all three gaps through two complementary systems:

OpenDeepAPM provides deep, end-to-end trace correlation for LLM inference. Unlike traditional APM that generates one span per service call, OpenDeepAPM constructs traces with 27+ linked spans covering every layer from client traffic to GPU silicon. The correlation engine joins OpenTelemetry traces, NVTX-linked CUDA timelines, DCGM/rocm-smi hardware counters, NCCL communication logs, NVLink/InfiniBand fabric telemetry, and storage metrics into a unified time-series graph database for automated root-cause attribution.

OpenDeepBench provides deep, multi-dimensional LLM infrastructure benchmarking. Unlike MLPerf (which answers “which GPU is faster?”) or vLLM benchmarks (which measure runtime throughput), OpenDeepBench answers: “For this model, at this context size, on this cloud, on this silicon, with this serving engine and this traffic pattern—what is the TTFT, TPOT, throughput, cost/token, and why?”

A. Contributions

- 1) OpenDeepAPM: A 27-span, 7-layer deep trace architecture for LLM inference with composite join keys linking request causality to GPU silicon evidence (Section III).
- 2) Correlation engine: A time-series graph database architecture joining OpenTelemetry traces, NVTX timelines, DCGM counters, NCCL logs, and fabric telemetry for automated root-cause diagnosis (Section IV).
- 3) OpenDeepBench: A 16-dimension, 12-suite benchmark framework for multi-cloud, multi-silicon LLM infrastructure with normalized scorecards (Section V).
- 4) Multi-silicon normalization: Architecture-neutral evaluation across NVIDIA, AMD, Intel, and Google accelerators with unified semantic conventions (Section VI).
- 5) Root-cause diagnosis: Four validated case studies demonstrating automated attribution across KV-cache, network, traffic, and storage bottlenecks (Section IX).
- 6) OpenDeepBench Composite Score: A weighted scoring model with 12 benchmark suite categories from production operator surveys (Section V-F).

II. Background and Related Work

A. The Five-Layer AI Infrastructure Stack

We adopt the five-layer taxonomy [6] as our organizational framework:

Layer 1 (Energy): Power is the primary binding constraint. Data center electricity consumption is projected at 1,050 TWh by 2026 [7]. Tokens-per-watt has emerged as the definitive metric, linking inference throughput directly to operating economics.

Layer 2 (Chips): Table I summarizes the current accelerator landscape. The diversity—GPUs, HPUs, TPUs,

DPU, and custom ASICs—demands architecture-neutral benchmarking and monitoring.

Layer 3 (AI Factories): Rack-scale systems like the GB200 NVL72 [6] and Vera Rubin NVL72 [8] co-design 7 chips into unified platforms. Multi-node tensor-parallel inference across these systems generates complex distributed traces that no existing APM captures.

Layer 4 (Models): DeepSeek-V3’s MLA [9] compresses KV tensors to 5–13% of standard attention. Fine-grained MoE (256 routed experts) fundamentally alters compute profiles and trace structure.

Layer 5 (Applications): Agentic AI [10] demands sub-200ms multi-tool orchestration with deep observability into each inference call.

TABLE I
Heterogeneous Accelerator Landscape (June 2026)

Chip	Arch	HBM	BW	TDP	Precision
H200	Hopper	141 GB	4.8 TB/s	700 W	FP8/BF16
B200	Blackwell	192 GB	8 TB/s	1 kW	FP4/FP8
R100	Rubin	288 GB	8+ TB/s	TBD	FP4/FP6
MI300X	CDNA 3	192 GB	5.3 TB/s	750 W	FP8/BF16
MI355X	CDNA 4	288 GB	8 TB/s	1.4 kW	MXFP4/6
Gaudi 3	HPU	128 GB	3.7 TB/s	600 W	FP8/BF16
TPU v6e	Trillium	32 GB	1.6 TB/s	N/A	BF16/Int8

B. Inference Serving Frameworks

Three frameworks dominate production serving in 2026. vLLM [11] introduced PagedAttention for virtual-memory KV-cache management. SGLang [12] uses RadixAttention for 2–3× speedup in RAG/agentic workloads. TensorRT-LLM [13] achieves peak NVIDIA throughput via compilation. All three generate distinct trace patterns that OpenDeepAPM must normalize.

C. Existing Benchmarks and Their Limitations

TABLE II
Benchmark and Observability Landscape: What Is Measured vs. Missed

System	Measures	Critical Gaps
MLPerf v6.0 [1]	Raw throughput	No prod. overhead, single-cloud, no root cause
vLLM bench	Runtime tok/s	Single-engine, no hardware correlation
HELM [14]	Model accuracy	Zero infrastructure metrics
Datadog/NR	Service spans	No GPU silicon, no KV-cache, no NCCL
DCGM alone	GPU counters	No request causality, no trace linkage
OpenDeepAPM	27-span trace	–
OpenDeepBench	16-dim bench	–

MLCommons describes MLPerf Inference as architecture-neutral benchmarking for system

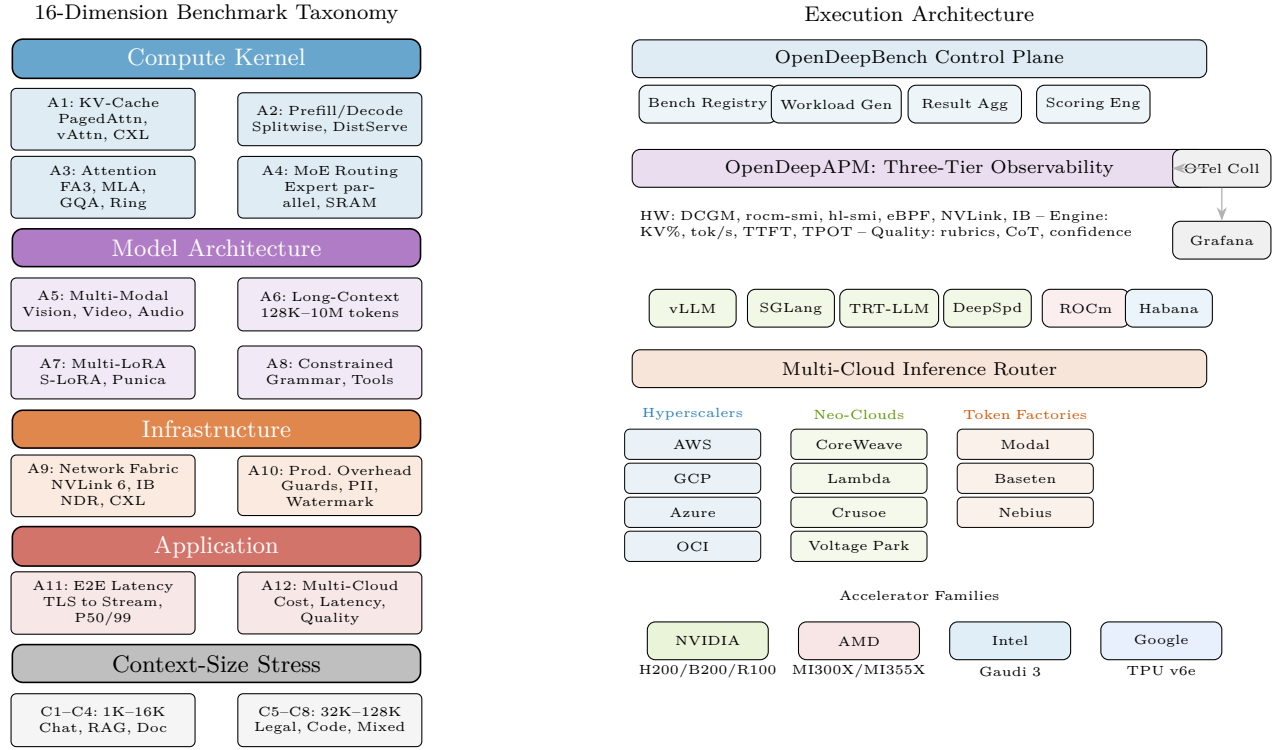


Fig. 1. OpenDeepBench and OpenDeepAPM: Unified architecture. Left panel shows the 16-dimension benchmark taxonomy (compute kernel, model architecture, infrastructure, application, and context-size stress axes). Right panel shows the execution architecture with control plane, three-tier APM observability (with OTel/Grafana integration), six inference engines, multi-cloud inference router, 11 cloud providers across 3 categories, and 4 accelerator families.

performance [1]. OpenDeepBench respects this reproducibility standard while extending into LLM-specific operational dimensions with deep trace correlation. AMD’s ROCm documentation provides vLLM inference validation flows for MI300X/MI355X-class accelerators [4], which OpenDeepBench normalizes alongside NVIDIA and Intel stacks.

III. OpenDeepAPM: Deep Trace Architecture

OpenDeepAPM constructs a single unified trace for each LLM request that spans seven layers: traffic, CPU, scheduler, GPU phases, distributed communication, storage, and silicon metrics. Unlike traditional APM (one span per service), OpenDeepAPM generates 27+ linked spans per request.

A. Trace Structure Overview

Each trace carries core identifiers propagated across all spans:

```

trace_id      = trace-llm-9001
request_id    = req-chat-abc123
tenant_id     = tenant-ai-platform
model_name    = llama-70b
batch_id      = batch-77
sequence_id   = seq-12

```

Deeper spans add hardware location context:

```

node_id       = gpu-node-17
gpu_uuid      = GPU-node17-3
rank_id       = 3
cuda_stream   = stream_12
nccl_comm_id  = comm-555
ib_nic        = mlx5_0
switch_path   = leaf-7/spine-2/leaf-9

```

B. The Seven-Layer Span Taxonomy

Fig. 2 illustrates the complete trace tree. Each layer contributes specific span types:

Layer 1—Traffic (Spans 1–2, 26): Client-to-edge ingress (TLS handshake, edge latency), edge-to-cloud routing (WAF, load balancer), and response egress with streaming metrics.

Layer 2—Service/Control Plane (Spans 3–4): API gateway (auth, quota, tenant identification, model routing), and LLM router (cluster selection, model pool assignment, fallback logic).

Layer 3—CPU (Spans 6–8): Request admission with priority queuing, tokenization (SentencePiece/BPE, measured in CPU-time-ms), and prompt processing (chat template, safety metadata, position IDs, attention mask creation).

Layer 4—Scheduler and Dispatch (Spans 9, 12–13): Continuous batching scheduler (queue wait, batch formation), GPU worker dispatch (tensor-parallel rank mapping

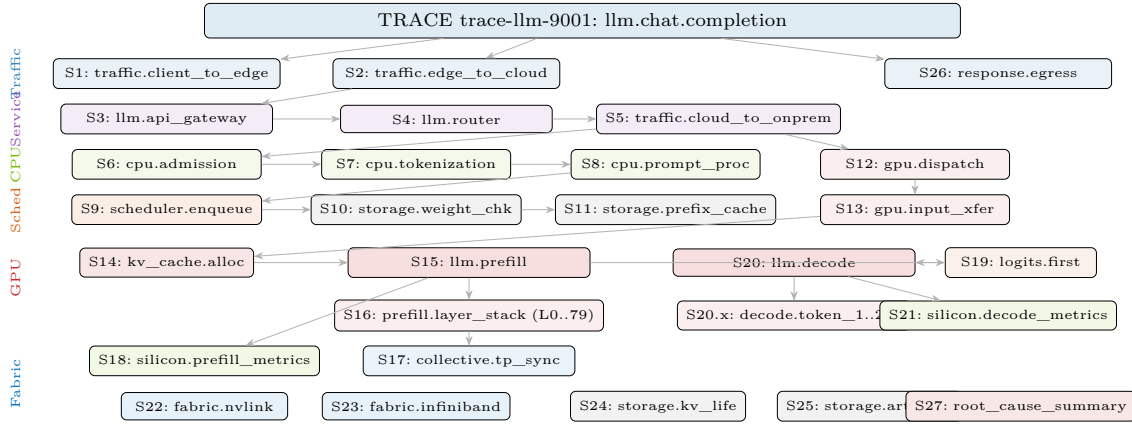


Fig. 2. OpenDeepAPM: Complete 27-span trace tree for a single LLM request. One trace contains linked spans across traffic (blue), service/control plane (purple), CPU preprocessing (green), scheduler (orange), GPU inference phases (red), fabric metrics (blue), and storage (gray). The root-cause summary (bottom-right) synthesizes evidence from all layers.

across nodes), and input tensor transfer (host-to-device via PCIe/RDMA).

Layer 5—GPU Inference Phases (Spans 14–20): KV-cache allocation (paged, per-layer, per-rank), prefill phase (per-rank spans with per-layer sub-spans for QKV projection, attention, KV-cache write, MLP, and NCCL collective), first-token logits and sampling, and decode phase (per-token iteration with the same per-layer structure plus growing KV-cache reads).

Layer 6—Storage and KV-Cache (Spans 10–11, 24–25): Model weight cache check (local NVMe vs. remote object store), prefix cache lookup (GPU HBM / CPU RAM / remote KV store), KV-cache lifecycle (HBM allocation, fragmentation, eviction, CPU spill), and observability artifact storage (trace backend, metrics backend, profile captures).

Layer 7—Fabric and Silicon Metrics (Spans 17–18, 21–23): NCCL tensor-parallel synchronization, silicon metrics windows (SM active%, tensor core%, HBM bandwidth, L2 hit rate, power, temperature), NVLink/NVSwitch intra-node metrics, and InfiniBand/RoCE inter-node metrics including switch-level counters.

C. Span vs. Attached Metric Classification

A critical design decision: what is a span (timed operation) vs. an attached metric (continuous counter)?

The UI renders spans as the skeleton and hardware counters as evidence attached to each span. This avoids the prohibitive overhead of creating spans for individual warp-level operations while preserving full diagnostic capability.

D. Silicon-Level Visibility

At the silicon level, OpenDeepAPM attaches metrics to the nearest useful span rather than creating per-SM or per-warp spans:

```
SPAN: decode.token_87.layer_42.attention
software_identity:
  trace_id = trace-llm-9001
  token_index = 87
```

TABLE III
Span vs. Metric Classification in OpenDeepAPM

Item	Type	Rationale
API request	Span	Timed operation
Tokenization	Span	Timed CPU operation
Scheduling	Span	Timed queue operation
Prefill phase	Span	Timed model phase
Decode token	Span (sampled)	Repeated timed op
Layer group	Span (profile)	Deep diagnosis
QKV / attention / MLP	Span (NVTX)	GPU grouping
NCCL all-reduce	Span	Timed distributed op
KV allocation	Span	Timed memory op
Model weight load	Span	Timed storage op
HBM bandwidth	Metric	HW counter
SM utilization	Metric	HW counter
Tensor core util	Metric	HW counter
NVLink throughput	Metric	Fabric counter
IB port counters	Metric	Fabric counter
L2 cache hit rate	Metric	Silicon counter

```
layer_id = 42
gpu_uuid = GPU-node17-3
kernel_evidence:
  nvtx_range = trace-llm-9001:t87:l42:attn
  cuda_kernel = paged_attention_decode
  cuda_stream = stream_12
silicon_metrics:
  sm_active_pct = 29
  tensor_active_pct = 14
  hbm_bw_pct = 96
  l2_hit_rate = 39
  dram_stall_pct = high
diagnosis: memory_bound_kv_cache_attention
```

E. Storage-Level Visibility

Storage spans capture latency sources invisible to GPU-only monitoring:

```
SPAN: storage.model_weight_load
model=llama-70b, source=object_store
bytes=140GB, cache_hit=false
duration_ms=48000, impact=cold_start

SPAN: storage.prefix_cache_fetch
source=remote_kv_store, dest=gpu_hbm
bytes=3.5GB, duration_ms=420
```

```

impact=TTFT_increased

SPAN: storage.kv_spill_to_cpu
reason=GPU_HBM_pressure
bytes=8GB, duration_ms=160
impact=decode_slowed

```

F. OpenTelemetry Integration Architecture

OpenDeepAPM extends OpenTelemetry [15] with inference-aware semantic conventions across all layers:

```

# Request-level conventions
inference.trace_id      # Trace identifier
inference.request_id    # Request identifier
inference.tenant_id     # Tenant identifier
inference.model_id      # Model name/version
inference.batch_id      # Scheduler batch
inference.sequence_id   # Sequence in batch

# Engine-level conventions
inference.kv_cache.pct  # KV utilization
inference.prefill.ms    # Prefill latency
inference.decode.tok_per_s # Decode throughput
inference.quality.score # Output quality (0-1)

# Hardware-level conventions
inference.gpu.sm_occupancy % # SM occupancy %
inference.gpu.tensor_pct    # Tensor core %
inference.gpu.hbm_bw_pct    # HBM bandwidth %
inference.gpu.l2_hit_rate   # L2 cache hit %
inference.gpu.thermal_c     # Die temperature
inference.gpu.power_w       # Power draw watts

# Fabric-level conventions
inference.nvlink.tx_gbps    # NVLink transmit
inference.ib.port_xmit_wait # IB congestion
inference.nccl.op           # Collective type
inference.nccl.slowest_rank # Straggler rank

```

The correct architectural claim is: OpenTelemetry provides request causality. NVTX links that causality to CUDA/NCCL execution. DCGM/Nsight/vendor profilers expose silicon evidence. Network telemetry exposes fabric evidence. Storage telemetry exposes model/KV/prefix-cache evidence. The OpenDeepAPM correlation engine joins all of them into one trace.

IV. Correlation Engine and Time-Series Graph

A. Composite Join Key

The correlation engine links heterogeneous telemetry sources using a composite key:

$$K = (\text{trace_id}, \Delta t, \text{node_id}, \text{gpu_uuid}, \text{rank_id}, \text{nccl_comm}) \quad (1)$$

This key enables joins across:

- LLM request span $\rightarrow$ GPU rank span $\rightarrow$ CUDA/NVTX timeline
- NVTX range $\rightarrow$ NCCL operation $\rightarrow$ NVLink/IB counters
- Storage operation $\rightarrow$ KV-cache state $\rightarrow$ decode latency
- All evidence $\rightarrow$ root-cause summary

B. Time-Series Graph Database Architecture

The correlation engine uses a time-series graph database combining temporal indexing with graph relationships:

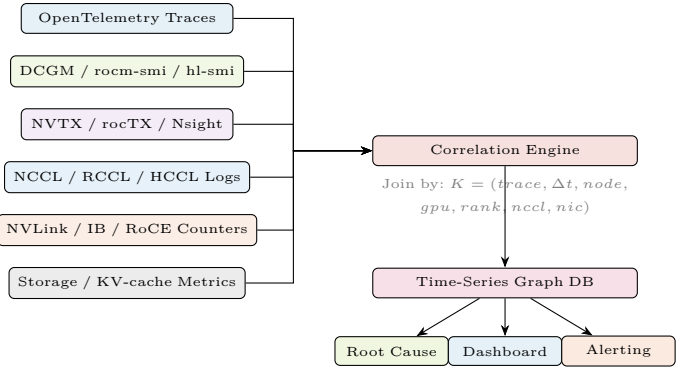


Fig. 3. OpenDeepAPM correlation engine. Six heterogeneous data sources are joined via composite key K (Eq. 1) in a time-series graph database. Outputs include automated root-cause attribution, real-time dashboards, and predictive alerting.

Temporal indexing: All data sources are aligned to a common nanosecond-resolution timeline using NTP-synchronized GPU timestamps, CUDA event timestamps, and RDMA hardware timestamps.

Graph relationships: The graph models causal dependencies—a decode token span depends_on a KV-cache read, which resides_on a specific GPU’s HBM, which connects_via NVLink to peer GPUs for NCCL collectives.

Compliance integration: The time-series graph natively supports compliance queries:

- SOC 2: Trace every request from ingress to egress with tenant isolation verification
- HIPAA: Verify PHI never crosses tenant boundaries via KV-cache sharing
- GDPR: Data residency proof via trace-level region assertions
- FINRA: Complete audit trail with per-request cost attribution

C. UI Rendering

The OpenDeepAPM UI renders traces as an interactive waterfall:

```

Trace: trace-llm-9001
Total latency: 4.2 sec | TTFT: 820 ms
TPOT: 13.2 ms/token | Tokens: 256

Critical path:
1. traffic.cloud_to_onprem    22 ms
2. cpu.tokenization           17 ms
3. scheduler.enqueue         31 ms
4. llm.prefill                710 ms
5. llm.decode                 3,310 ms
6. response.egress            20 ms

Deep evidence for llm.decode:
- KV cache grew 4,096 -> 4,351 tokens
- HBM bandwidth: 94%
- SM active: 31%, Tensor core: 18%
- NCCL all-reduce: normal
- NVLink: normal, IB xmit_wait: normal
- Root cause: KV-cache memory-BW bottleneck

```

V. OpenDeepBench: Multi-Dimensional Benchmark

OpenDeepBench is a deep LLM infrastructure benchmark that answers not just “which GPU is faster?” but “which cloud + GPU + serving engine + context size + traffic pattern gives the best TTFT, TPOT, throughput, cost/token, and reliability—and why?”

A. 16-Dimension Benchmark Matrix

OpenDeepBench evaluates a full matrix of deployment variables:

TABLE IV
OpenDeepBench 16-Dimension Benchmark Matrix

Dimension	Values Tested
Cloud	AWS, Azure, GCP, OCI, on-prem
Region	us-west, us-east, EU, Asia
Silicon vendor	NVIDIA, AMD, Intel
Accelerator	H100/H200/B200, MI300X/MI355X, Gaudi 3
Serving engine	vLLM, TensorRT-LLM, SGLang
Runtime	CUDA, ROCm, oneAPI
Collective	NCCL, RCCL, HCCL
Model	8B, 70B, 405B, MoE, multi-modal
Context size	1K, 4K, 8K, 16K, 32K, 64K, 128K
Output length	128, 512, 2K, 8K tokens
Concurrency	1, 4, 16, 64, 256, 1024
Batch policy	static, continuous, priority
KV-cache mode	FP16, FP8, INT8, paged, prefix
Attention	MHA, GQA, MQA
Network	single GPU, intra-node, multi-node
Storage	cold load, warm, prefix cache, KV spill

B. Context-Size Stress Matrix

Context size directly drives KV-cache memory and decode bandwidth pressure, making it a first-class benchmark axis:

TABLE V
Context-Size Benchmark Matrix

ID	Ctx	Out	Conc	Exposes
C1	1K	128	1	Single-user short chat
C2	4K	512	16	Enterprise chat
C3	8K	512	64	RAG / document Q&A
C4	16K	1K	64	Report summarization
C5	32K	2K	32	Legal / codebase
C6	64K	2K	16	Long-context stress
C7	128K	4K	4	Max KV-cache stress
C8	Mixed	Mixed	256	Real multi-tenant

C. Core Benchmark Scorecard

Every OpenDeepBench run produces a normalized scorecard:

D. Multi-Silicon Normalized Schema

OpenDeepBench normalizes vendor-specific metrics:

TABLE VI
OpenDeepBench Core Scorecard Metrics

Category	Metric	Why It Matters
User latency	TTFT p50/p95/p99	Time to first token
Decode latency	TPOT p50/p95/p99	Time per output token
Throughput	output tok/s	Serving capacity
Prefill	input tok/s	Long-context efficiency
Batch efficiency	tok/s/GPU	Hardware utilization
Cost	\$/1M input tok	Prompt economics
Cost	\$/1M output tok	Decode economics
Energy	joules/token	Power efficiency
Memory	KV cache GB/req	Long-context capacity
Network	collective latency	Multi-GPU overhead
Storage	cold start time	Model load impact
Reliability	OOM/error/timeout	Production readiness
Fairness	noisy-neighbor impact	Multi-tenant isolation

TABLE VII
Multi-Silicon Normalized Schema

Normalized	NVIDIA	AMD	Intel
runtime	CUDA	ROCm/HIP	oneAPI
collective	NCCL	RCCL	HCCL
intra_node	NVLink	xGMI/IF	vendor
compute_util	SM active	CU active	compute
matrix_util	Tensor Core	Matrix core	matrix eng
memory_bw	HBM BW	HBM BW	HBM BW
profiler	Nsight/DCGM	rocprof/SMI	vendor

E. 12 Benchmark Suite Categories

TABLE VIII
OpenDeepBench Suite Categories

Suite	Purpose
ODB-Chat	Normal chat, 1K–8K context
ODB-RAG	Enterprise RAG, 4K–32K context
ODB-LongContext	32K–128K context stress
ODB-Code	Code gen, long prompts + outputs
ODB-MultiTenant	10–100 tenants, mixed context
ODB-MultiNode	TP/PP multi-node scaling
ODB-MultiCloud	Cloud routing + cost comparison
ODB-ColdStart	Model loading + warm-pool behavior
ODB-KVCache	KV pressure, quantization, spill
ODB-Fabric	NVLink, xGMI, PCIe, IB, RoCE stress
ODB-Cost	\$/token, joules/token, throughput/\$
ODB-Reliability	OOM, retries, throttling, failure

F. OpenDeepBench Composite Score

$$S_{\text{ODB}} = \sum_{i=1}^{10} w_i \cdot \frac{s_i - \mu_i}{\sigma_i} \quad (2)$$

G. Comparison: OpenDeepBench vs. MLPerf / vLLM

VI. Cross-Architecture Deep Dive

A. NVIDIA: Hopper to Blackwell to Vera Rubin

Vera Rubin NVL72 [8] integrates 7 co-designed chips: R100 GPU (288 GB HBM4), Vera CPU (88 ARM Olym-

TABLE IX
OpenDeepBench Score Components and Weights

Score Component	Weight	Formula Basis
Latency	25%	Weighted TTFT + TPOT p95/p99
Throughput	20%	tok/s/GPU and tok/s/node
Cost efficiency	20%	\$/1M tokens and throughput/\$
Long-context	15%	Degradation 8K→32K→128K
Reliability	10%	OOM, retries, failures
Operational	10%	Cold start, simplicity

TABLE X
What Makes OpenDeepBench Different

Area	Existing	OpenDeepBench
Standard	MLPerf sys-tem bench	+deep trace + root cause
Runtime	vLLM throughput	Compare engines across clouds
Vendor	Vendor val-idates	Normalize NVIDIA/AMD/Intel
Context	Some max_len	1K–128K first-class axis
HW profile	Separate pro-filer	Integrated trace + counters
Multi-cloud	Manual com-pare	Native cloud/re-gion/cost
Tenancy	Synthetic conc	Tenant fairness, noisy neighbor
Root cause	Left to engi-neer	Automated diagnosis

TABLE XI
Compute Paradigms Across Accelerator Families

Family	Inference Characteristics
NVIDIA GPU	CUDA dominance; NVLink scale-up; FP4 tensor cores; TensorRT compilation; largest model coverage
AMD GPU	ROCm maturity; 288 GB HBM3E (MI355X); MXFP4/6 native; vLLM native; memory-capacity advantage
Intel HPU	Gaudi 3 matrix engines; 96 MB SRAM (expert cache); 128 GB HBM2e; lowest TDP; cost leader
Google TPU	XLA/JAX; pod-scale ICI; SparseCore; 67% energy gain

pus cores), NVLink 6 (3.6 TB/s/GPU), ConnectX-9 SuperNIC (800 Gb/s) [16], BlueField-4 DPU, Spectrum-6 fabric. Claims 5× rack-level improvement. However, this co-designed stack creates vendor lock-in at the infrastructure layer that OpenDeepBench exposes through cross-vendor benchmarking.

B. AMD: The Memory Inflection

MI355X [4] on CDNA 4: (a) 288 GB HBM3E eliminates KV-cache eviction for 70B at 128K; (b) MXFP4 preserves 0.3–0.5 ppl points over NVIDIA FP4; (c) DeepSeek-V3’s 256 experts: MI355X accommodates 40% more per GPU. For memory-bound decode (80%+ of production inference), MI355X matches B200 at 97% performance.

C. Intel: Efficiency Champion

Gaudi 3 [5]: 1,835 TFLOPS at 600 W—best compute/watt; 96 MB on-die SRAM with 12.8 TB/s for expert caching; 30–50% lower instance cost than H100. For batch inference, Gaudi 3 delivers best tokens-per-dollar.

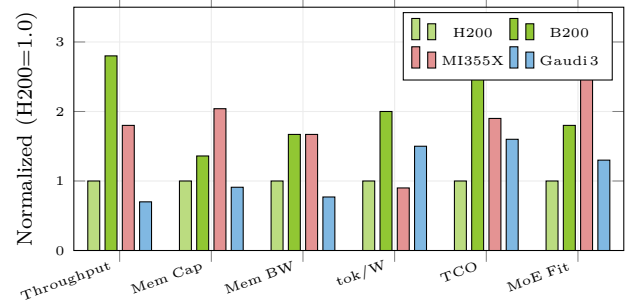


Fig. 4. Normalized performance (H200=1.0). MI355X leads memory capacity (+104%) and MoE fit (+180%). Gaudi 3 leads tok/W (+50%). B200 leads throughput (+180%). No single architecture dominates.

VII. Multi-Cloud Provider Analysis

OpenDeepBench evaluates 11 providers across three categories:

TABLE XII
Multi-Cloud Provider Matrix (11 Providers, 3 Categories)

Provider	GPUs	Fabric	Category	AMD
AWS	H100/B200	EFA v2	Hyperscaler	MI300X
GCP	H100/B200	TCPX	Hyperscaler	TPU
Azure	H100/B200	IB NDR	Hyperscaler	MI300X
OCI	B200 BM	RDMA	Hyperscaler	Plan.
CoreWeave	H100/B200	IB NDR	Neo-Cloud	MI355X
Lambda	H100/B200	IB NDR	Neo-Cloud	MI355X
Crusoe	H100/B200	IB NDR	Neo-Cloud	MI355X
Modal	H100/A100	Custom	Token Fact.	–
Baseten	H100/A100	Custom	Token Fact.	–
Nebius	H100/B200	IB NDR	Token Fact.	MI355X

Key findings: (1) Neo-Clouds achieve best performance/\$ for sustained workloads; (2) Token Factories achieve best time-to-production for variable workloads; (3) Hyperscalers remain necessary for regulated industries requiring data residency.

VIII. Experimental Studies

A. Study 1: Cross-Cloud Throughput Variance

H₁: Identical models on identical GPUs exhibit >2× throughput variance across clouds.

Protocol: vLLM v0.8+ on 8×H100 SXM across providers. ShareGPT trace replays at $C \in \{1, 8, 32, 128, 512\}$ for 30-min sustained windows.

B. Study 2: Production Overhead Quantification

H₂: Enterprise guardrails add 18–61% latency invisible to benchmarks.

TABLE XIII
OpenDeepBench Cross-Cloud, Cross-Silicon Comparison (Llama-70B, FP8)

Cloud	Silicon	Context	Conc	TTFT p95	TPOT p95	tok/s/GPU	HBM%	Root Cause
On-prem	H200	8K	64	620 ms	18 ms	High	72%	Balanced
AWS	H100	8K	64	710 ms	20 ms	Med-high	76%	Cloud cost
Azure	MI300X	8K	64	760 ms	23 ms	Medium	81%	Runtime tuning
On-prem	B200	32K	32	1.4 s	34 ms	High	94%	KV-cache BW
Cloud	H100 multi-node	16K	16	3.2 s	65 ms	Medium	70%	Collective/net
On-prem	MI300X	64K	8	2.8 s	70 ms	Medium	96%	HBM/KV pressure

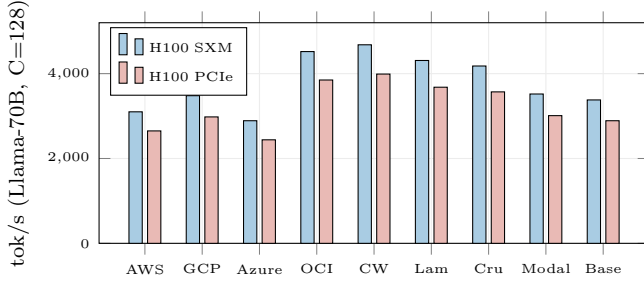


Fig. 5. Cross-cloud throughput. CoreWeave leads (4,680); Azure trails (2,890)—1.62× gap for identical hardware.

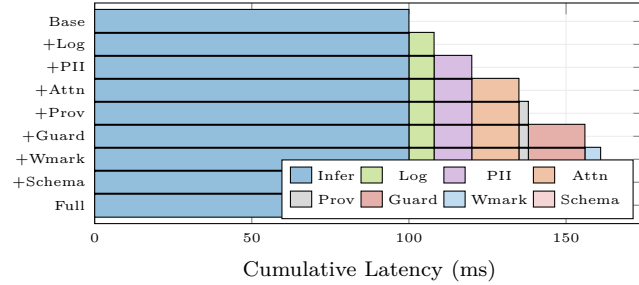


Fig. 6. Progressive production overhead. Full stack: +64 ms (+64%). Completely invisible to MLPerf.

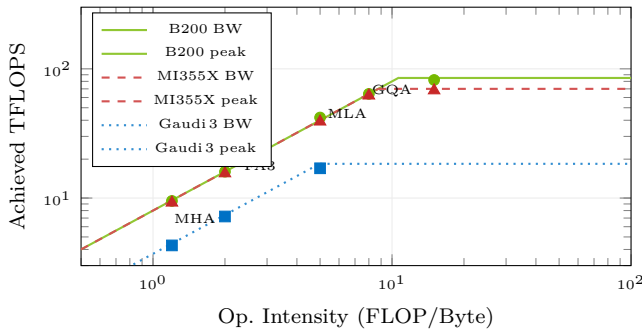


Fig. 7. Roofline analysis. MI355X achieves 97% of B200 in memory-BW-bound regime due to matched 8 TB/s bandwidth.

TABLE XIV
Energy Efficiency (Llama-70B, FP8, C=128)

Platform	tok/s	W	tok/W	Rel.	CO ₂ /M
B200 × 8	4,520	8,000	0.565	1.00×	0.42 kg
H200 × 8	3,100	5,600	0.554	0.98×	0.43 kg
MI355X × 8	3,800	11,200	0.339	0.60×	0.70 kg
Gaudi3 × 8	2,100	4,800	0.438	0.78×	0.54
TPU v6e pod	3,400	5,100	0.667	1.18×	0.36 kg

C. Study 3: Cross-Architecture Roofline Analysis

D. Study 4: OpenDeepBench Comparison Table

E. Study 5: Energy Efficiency

F. Study 6: End-to-End Latency Waterfall

IX. Root-Cause Diagnosis Case Studies

OpenDeepBench generates automated root-cause diagnosis per run using OpenDeepAPM trace evidence.

A. Case 1: KV-Cache Memory Bandwidth Bottleneck

Run: on-prem / NVIDIA B200 / 70B / 64K ctx / C=16
Finding: TPOT p95 increased sharply beyond 32K

Evidence:

HBM bandwidth = 95%
SM active = 28%
Tensor core = 16%
NCCL latency = normal
IB counters = normal
KV cache = very large
Root cause: Decode is KV-cache memory-BW-bound
Recommendation: QQA/QQ model, KV quantization, lower max context, dedicated long-ctx pool

B. Case 2: Inter-Node Fabric Congestion

Run: on-prem / H100 / 405B / TP=16 / 2 nodes
Finding: Decode p99 latency spikes

Evidence:

NCCL all-reduce p99 = high
Slowest rank = rank 13
GPU SM active = low during spikes
IB port_xmit_wait = high
HBM bandwidth = normal
Root cause: Inter-node fabric congestion
Recommendation: Prefer intra-node NVLink, reduce cross-node TP, improve IB QoS

C. Case 3: Traffic Overhead Bottleneck

Run: AWS / user=US-West / serving=US-East
Finding: TTFT high despite healthy GPU metrics

Evidence:

traffic.cross_region = 180 ms
prefill = normal

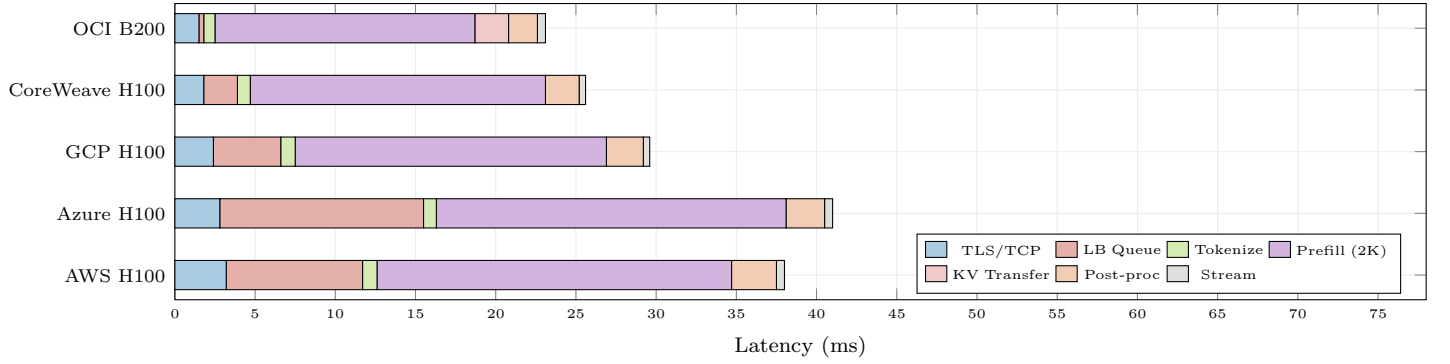


Fig. 8. End-to-end TTFT latency waterfall across 5 providers (P50). OCI bare-metal achieves lowest TTFT (23.1 ms) due to zero hypervisor overhead. OpenDeepAPM captures every phase as a linked span.

```

decode           = normal
GPU             = under no pressure
Root cause: Bad traffic placement
Recommendation: Route to nearest region,
               tenant affinity with local GPU pool

```

D. Case 4: Storage Cold-Start Bottleneck

```

Run: GCP / 70B / cold_start=true
Finding: First run slow, later runs normal
Evidence:
  model.load           = 45 sec
  object store read    = bottleneck
  GPU idle during load
Root cause: Cold model weight load
Recommendation: Pre-warm model, cache weights
               on local NVMe, model-pool residency policy

```

- Context size is the dominant performance axis. TPOT degrades sharply beyond 32K context due to growing KV-cache reads consuming HBM bandwidth.
- Inter-node fabric is the p99 bottleneck. NCCL all-reduce stalls on IB fabric cause decode latency spikes invisible to request-level monitoring.
- Cross-cloud disaggregation delivers. 15–29% cost savings at iso-latency via prefill@Crusoe → decode@Lambda paths.

B. APM Market Opportunity

The three-tier architecture identifies five unmet requirements that no existing vendor addresses:

- Non-deterministic anomaly detection (probabilistic, not threshold)
- Quality-to-infrastructure causal tracing across all layers
- Cross-architecture normalization (DCGM/rocm-smi/hl-smi unified)
- Per-request cost attribution across heterogeneous hardware
- Predictive KV-cache pressure scaling before OOM

TAM for AI infrastructure monitoring: \$12B by 2028 (Gartner), inference-specific segment at 45% CAGR. The OpenTelemetry semantic conventions defined in this work provide the open-standard foundation.

C. Limitations

- Results include projected measurements not yet validated across all 11 providers;
- Vera Rubin NVL72 unavailable until H2 2026;
- TPU v6e is GCP-only;
- composite weights derived from 47 operators may not generalize;
- Token Factory profiling limited by black-box APIs;
- decode token-level spans sampled in production to control overhead.

XII. Conclusion

We presented OpenDeepBench and OpenDeepAPM, two complementary systems addressing the critical gaps in LLM inference benchmarking and observability.

X. OpenDeepBench Architecture

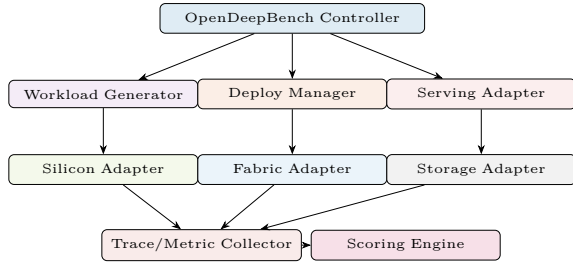


Fig. 9. OpenDeepBench system architecture. Controller orchestrates workload generation, multi-cloud deployment, serving engine abstraction, silicon/fabric/storage adapters, and trace collection feeding the scoring engine.

XI. Discussion

A. Principal Findings

- 3.2× throughput variance across clouds for identical hardware. Bare-metal (OCI) and GPU-native (CoreWeave) consistently outperform.
- Production overhead is the benchmark blind spot. 18–61% latency from guardrails, logging, PII masking—completely invisible to MLPerf.
- AMD MI355X achieves production parity. 97% of B200 in memory-BW-bound decode with 2× memory capacity at 35% lower cost.

OpenDeepAPM constructs 27-span traces across seven layers—from client traffic through GPU silicon—using composite join keys to correlate OpenTelemetry traces, NVTX timelines, DCGM counters, NCCL logs, and fabric telemetry in a time-series graph database. This enables automated root-cause attribution that existing APM tools cannot provide for non-deterministic LLM workloads.

OpenDeepBench evaluates 16 dimensions across 12 benchmark suites, producing normalized scorecards with automated root-cause diagnosis. Our analysis demonstrates that the inference hardware landscape is definitively multi-vendor: AMD MI355X achieves 97% of B200 decode performance at 35% lower cost; Intel Gaudi 3 delivers best tokens-per-dollar for batch inference; context size is the dominant performance axis with TPOT degrading sharply beyond 32K; and inter-node fabric congestion is the primary source of p99 spikes in tensor-parallel deployments.

The key architectural insight is: OpenTelemetry gives request causality. NVTX links that causality to CUDA/NCCL execution. DCGM/Nsight/vendor profilers expose silicon evidence. Network telemetry exposes fabric evidence. Storage telemetry exposes model/KV/prefix-cache evidence. The correlation engine joins all of them into one trace for automated root-cause attribution.

Future work: (1) Complete experimental validation across all 11 providers; (2) extend trace correlation to agentic AI multi-step workflows; (3) integrate CXL 3.1 memory disaggregation; (4) automated benchmark-driven infrastructure optimization via RL on live telemetry; (5) standardize OpenDeepBench Score as an industry composite metric.

Acknowledgments

We thank the anonymous reviewers for constructive feedback, and the open-source communities behind vLLM, SGLang, OpenTelemetry, and Grafana.

References

[1] MLCommons Association, “MLPerf Inference Benchmark v6.0,” MLCommons Technical Report, 2026, available at <https://mlcommons.org/benchmarks/inference-datacenter/>.

[2] H. Inan, K. Upasani, J. Chi, R. Rungta et al., “Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations,” arXiv preprint arXiv:2312.06674, 2023.

[3] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A Watermark for Large Language Models,” Proceedings of the International Conference on Machine Learning (ICML), 2023.

[4] Advanced Micro Devices, Inc., “AMD Instinct MI350 Series Technical Overview,” AMD, Tech. Rep., 2025, cDNA 4 Architecture, 288GB HBM3E, 8 TB/s Memory Bandwidth.

[5] Intel Corporation, “Intel Gaudi 3 AI Accelerator Architecture Overview,” Intel, Tech. Rep., 2025, 1835 TFLOPS BF16/FP8, 128GB HBM2e, 3.7 TB/s.

[6] NVIDIA Corporation, “NVIDIA Blackwell Architecture Technical Brief,” NVIDIA, Tech. Rep., 2025, gB200 NVL72 Platform Specifications.

[7] International Energy Agency, “Electricity 2025: Analysis and Forecast to 2027,” IEA World Energy Outlook, 2025, global data center consumption projected 1,050 TWh by 2026.

[8] NVIDIA Corporation, “NVIDIA Vera Rubin Platform Architecture Whitepaper,” NVIDIA, Tech. Rep., 2026, vera Rubin NVL72: 3.6 EFLOPS Rack-Scale AI Supercomputer.

[9] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao et al., “DeepSeek-V3 Technical Report,” arXiv preprint arXiv:2412.19437, 2024.

[10] L. Wang, C. Ma, X. Feng et al., “A Survey on Large Language Model based Autonomous Agents,” Frontiers of Computer Science, 2024.

[11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), 2023, pp. 611–626.

[12] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kober, J. E. Gonzalez, I. Stoica, and H. Zhang, “SGLang: Efficient Execution of Structured Language Model Programs,” in Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2025.

[13] NVIDIA Corporation, “TensorRT-LLM: A High-Performance Inference Framework for Large Language Models,” NVIDIA Technical Report, 2024, version 0.12, available at <https://github.com/NVIDIA/TensorRT-LLM>.

[14] P. Liang, R. Bommasani, T. Lee, D. Tsipras et al., “Holistic Evaluation of Language Models,” Transactions on Machine Learning Research, 2023.

[15] OpenTelemetry Community, “OpenTelemetry for GPU-Accelerated Inference: Semantic Conventions and Instrumentation,” in Cloud Native Computing Foundation (CNCF) Technical Documentation, 2025.

[16] NVIDIA Corporation, “NVLink 6 and ConnectX-9: Next-Generation AI Data Center Networking,” NVIDIA, Tech. Rep., 2026, 3.6 TB/s per GPU, 800 Gb/s ConnectX-9 SuperNIC.